UNIVERSITY OF TEHRAN

Electrical and Computer Engineering Department

Computer Aided-Design

CA_4

Name:

Mahdi Naieni 810101536

Amin Aghakasiri 810101381

Introduction:

In this exercise, we aim to design an initial sample of the processing element (PE) structure of the Eyeriss accelerator. The buffer module implemented in the previous exercise is required for this implementation.

We are already familiar with convolutional neural networks (CNNs) and the Eyeriss accelerator. First, the architecture of this accelerator and its computational process are explained in detail. Then, the details of each part of the processing element (PE) that needs to be implemented are introduced along with relevant notes.

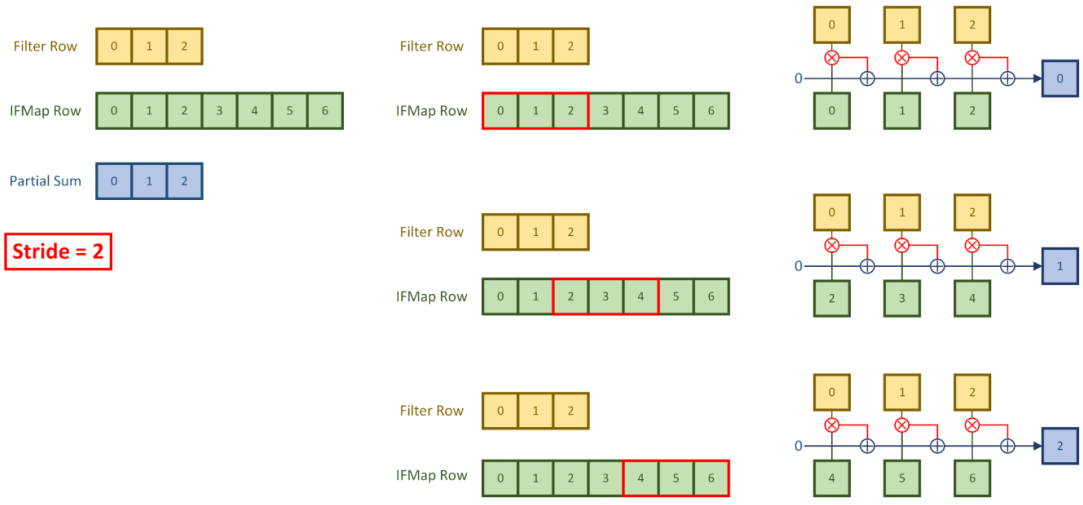
The overall structure of Eyeriss consists of a 14x12 array of processing elements (PEs), which are connected through a structure called Network-on-Chip (NoC) to a global buffer or neighboring units. Each processing element is capable of performing 1D convolution. Specifically, a PE receives a single-row filter matrix and a single-row input matrix, and it produces the convolution result in multiple steps.

A 2D or 3D convolution can be viewed as a series of 1D convolutions. Therefore, with a single processing element, these convolutions can also be performed. However, to speed up computations and reduce processing load, multiple processing elements are used together to carry out the overall operations in the Eyeriss accelerator.

1D convolution:

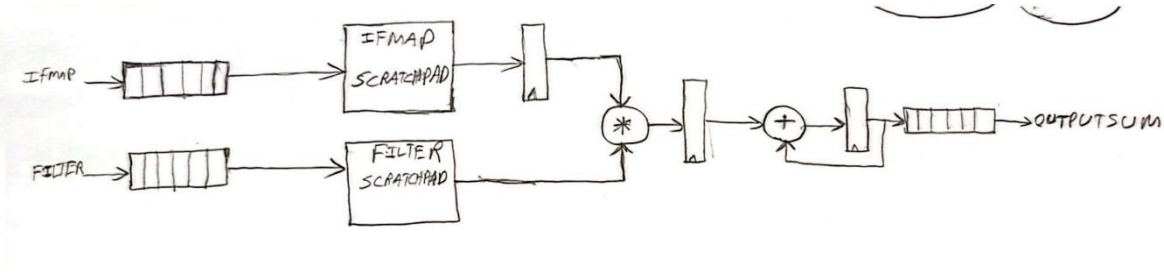
The steps for performing 1D convolution are as follows: First, a sliding window the size of the filter is placed over the input row. The elements within this window are multiplied by their corresponding elements in the filter, and the results are summed together. This sum forms the first output value, often referred to as the partial sum.

Next, the window shifts forward based on a value called the stride, which determines how many positions the window moves at each step. In the given example, the stride is 2, meaning the window shifts forward by two positions at each step. This process repeats, calculating the subsequent output values by performing the same multiplication and summation for the new window positions.



\

Data path:



This diagram represents the datapath of a Processing Element (PE) for convolution operations in a deep learning accelerator:

1. Inputs:
    - `IFMap` (Input Feature Map) and `Filter` data are loaded into two separate scratchpad memories:
        - IFMap ScratchPad (Register File, 12x16b)
        - Filter ScratchPad (SRAM, 24x16b)
2. Multiplication:
    - Data from both memories is fed into a 1-stage pipelined multiplier.
3. Accumulation:
    - The multiplier output is fed into an adder for accumulation (partial sum computation).
4. Control:
    - The PE Control Unit manages data flow, computations, and accumulation.
5. Output:
    - The accumulated partial sums (`Psum`) are written back to memory or passed to the next processing stage.

This design efficiently supports matrix multiplications in convolution operations, key for deep learning accelerators.

```
module datapath #(
    parameter WIDTH = 4,
    parameter SIZE_IFMAP = 4,
    parameter SIZE_FILTER_SRAM = 4,
    parameter ADDR_WIDTH_IFMAP = $clog2(SIZE_IFMAP),
    parameter ADDR_WIDTH_FILTER = $clog2(SIZE_FILTER_SRAM)
) (
    input clk,
    input rst,
    input rst_reg,
    input en_reg,

    input ifmap_read_en,
    input ifmap_write_en,
    input [ADDR_WIDTH_IFMAP-1:0] ifmap_address_read,
    input [ADDR_WIDTH_IFMAP-1:0] ifmap_address_write,
    input [WIDTH-1:0] ifmap_in,

    input filter_read_en,
    input filter_write_en,
    input [ADDR_WIDTH_FILTER-1:0] filter_address_read,
    input [ADDR_WIDTH_FILTER-1:0] filter_address_write,
    input [WIDTH-1:0] filter_in,

    output  [WIDTH-1:0] out
);

    wire add_co;
    wire [WIDTH-1:0] ifmap_out, filter_out, mult_out, add_out, reg_out;

    IFMapScratchPad #(
        .WIDTH(WIDTH),
        .SIZE(SIZE_IFMAP),
        .ADDR_WIDTH(ADDR_WIDTH_IFMAP)
    ) ifmap_reg (
        .clk(clk),
        .read_en(ifmap_read_en),
        .write_en(ifmap_write_en),
        .address_read(ifmap_address_read),
        .address_write(ifmap_address_write),
        .data_in(ifmap_in),
        .data_out(ifmap_out)
    );

    FilterScratchPad #(
        .WIDTH(WIDTH),
        .SIZE(SIZE_FILTER_SRAM),
        .ADDR_WIDTH(ADDR_WIDTH_FILTER)
    ) filter_sram (
        .clk(clk),
        .read_en(filter_read_en),
        .write_en(filter_write_en),
        .address_read(filter_address_read),
        .address_write(filter_address_write),
        .data_in(filter_in),
        .data_out(filter_out)
    );

    multiplier #(.WIDTH(WIDTH)) mult (.num1(filter_out), .num2(ifmap_out), .out(mult_out));

    adder #(.WIDTH(WIDTH)) add (.num1(mult_out), .num2(reg_out), .out(add_out), .co(add_co));

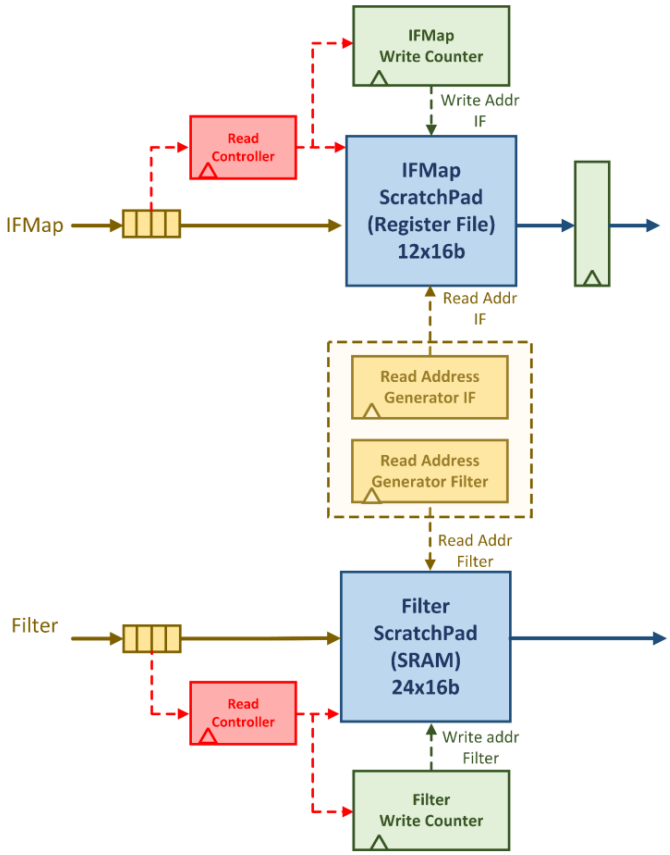
    final_register #(
        .WIDTH(WIDTH)
    ) final_reg(
        .clk(clk),
        .rst(rst | rst_reg),
        .en(en_reg),
        .pin(add_out),
        .pout(reg_out)
    );

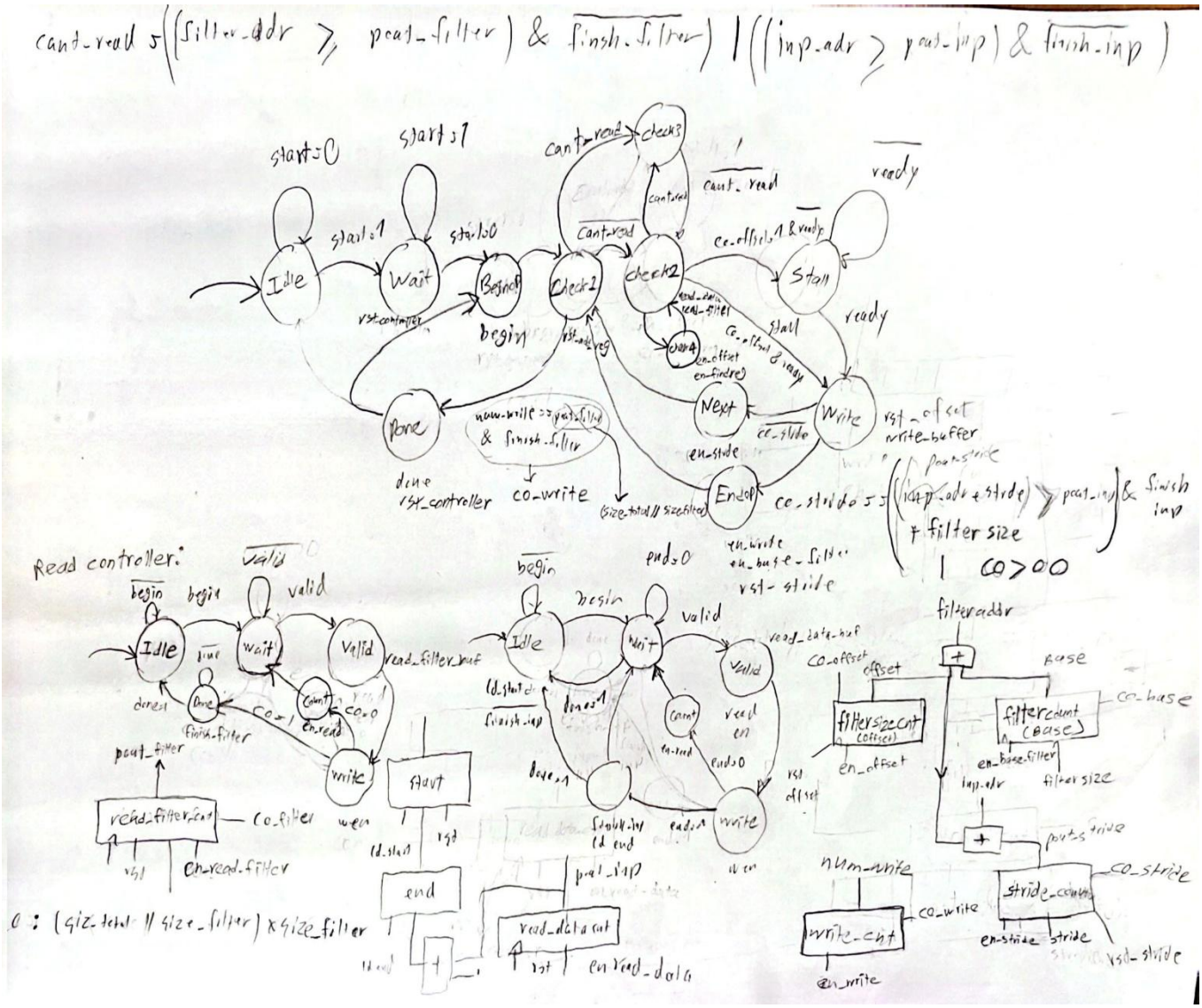
    assign out = reg_out;
endmodule
```

Controller:

To automate and consistently transfer data from buffers to the ScratchPads, a Buffer Read Controller is required. This controller generates control signals to manage data reads from the buffer and writes to the corresponding ScratchPads. The controller ensures that data is transferred whenever writing to the ScratchPad is allowed and data is available in the FIFO buffer. The figure illustrates the connections between control units, buffers, and ScratchPads.

To transfer the Psum (partial sum) to the output buffer after computations are complete, another control unit is required. This unit checks if the done signal (indicating the Psum computation is finished) is received from the main controller and if the output buffer can accept new data. If so, it writes the Psum to the buffer. If the buffer cannot accept data, the unit outputs a stall signal to the main controller to prevent overwriting or loss of data.




cant_read = ((filter_addr ≥ pout_filter) & finish_filter) | ((inp_adr ≥ pout_inp) & finish_inp)
start=0
start=1
start=10
Idle
Wait
Begin
Check1
Check2
Check3
Stall
Read
Next
Write
Done
End of
begin
ready
rst_controller
co_write
Read controller:
valid
Idle
wait
Valid
Count
Done
read_filter_buf
pout_filter
read_filter_cnt
co_filter
en_read_filter
Addr
end
read_data_cnt
filteraddr
offset
base
filtersize cnt (offset)
filter cnt (base)
co_base
en_offset
en_base_filter
filter size
inp_adr
pout_stride
num_write
write_cnt
co_write
stride_cnt
co_stride
en_stride
stride
en_write
rst_stride
0 : (size_total // size_filter) x size_filter

```
`define Idle 4'b0000
`define Wait 4'b0001
`define BeginOp 4'b0010
`define Check1 4'b0011
`define Check2 4'b0100
`define Check3 4'b0101
`define Check4 4'b0110
`define Stall 4'b0111
`define Write 4'b1000
`define Next 4'b1001
`define EndOp 4'b1010
`define Done 4'b1011

module main_controller (clk, rst, start, cant_read, ready, co_offset, co_stride, co_write,
                        beginop, rst_controller, rst_add_reg, read_data, read_filter , en_offset, en_final_reg, en_stride, stall, rst_stride, rst_offset, write_buffer, en_write, en_base_filter, done);

    input clk, rst, start, cant_read, ready, co_offset, co_stride, co_write;

    output beginop, rst_controller, rst_add_reg, read_data, read_filter, en_offset, en_final_reg, en_stride, stall, rst_stride, rst_offset, write_buffer, en_write, en_base_filter, done;

    reg beginop, rst_controller, rst_add_reg, read_data, read_filter , en_offset, en_final_reg, en_stride, stall, rst_stride, rst_offset, write_buffer, en_write, en_base_filter, done;

    reg [3:0] ps, ns;
    always @(posedge clk) begin
        if(rst)
            ps <= `Idle;
        else
            ps <= ns;
    end

    always @(ps, start, cant_read, ready, co_offset, co_stride, co_write) begin
        case(ps)
            `Idle: ns = start ? `Wait : `Idle;
            `Wait: ns = start ? `Wait : `BeginOp;
            `BeginOp: ns = `Check1;
            `Check1: ns = co_write ? `Done :
                          cant_read ? `Check3 :
                          `Check2;
            `Check2: ns = (co_offset == 1'b1 && ~ready) ? `Stall :
                          (co_offset == 1'b1 && ready) ? `Write :
                          cant_read ? `Check3 :
                          `Check4;
            `Check3: ns = ~cant_read ? `Check2 : `Check3;
            `Check4: ns = `Check2;
            `Stall: ns = ready ? `Write : `Stall;
            `Write: ns = co_stride ? `EndOp : `Next;
            `Next: ns = `Check1;
            `EndOp: ns = `Check1;
            `Done: ns = `BeginOp;
            default: ns = `Idle;
        endcase
    end

    always @(ps) begin
        {beginop, rst_controller, rst_add_reg, read_data, read_filter , en_offset, en_final_reg, en_stride, stall, rst_stride, rst_offset, write_buffer, en_write, en_base_filter, done} = 15'b0;
        case(ps)
            `Idle: ;
            `Wait: {rst_controller} = 1'b1;
            `BeginOp: {beginop} = 1'b1;
            `Check1: {rst_add_reg} = 1'b1;
            `Check2: {read_data, read_filter} = 2'b11;
            `Check3: ;
            `Check4: {en_offset, en_final_reg} = 2'b11;
            `Stall: {stall} = 1'b1;
            `Write: {rst_offset, write_buffer} = 2'b11;
            `Next: {en_stride} = 1'b1;
            `EndOp: {en_write, en_base_filter, rst_stride} = 3'b111;
            `Done: {done, rst_controller} = 2'b11;
        endcase
    end

endmodule
```

Testbenches:

```
module tb_read_filter_counter;

    parameter TOTAL_WIDTH = 8;
    parameter FILTER_SIZE_WIDTH = 4;
    reg clk, rst, ld, en;
    reg [TOTAL_WIDTH-1:0] pin;
    reg [TOTAL_WIDTH-1:0] size_total;
    reg [FILTER_SIZE_WIDTH-1:0] size_filter;
    wire cout;
    wire [TOTAL_WIDTH-1:0] pout;

    read_filter_counter #(
        .TOTAL_WIDTH(TOTAL_WIDTH),
        .FILTER_SIZE_WIDTH(FILTER_SIZE_WIDTH)
    ) uut (
        .clk(clk),
        .rst(rst),
        .ld(ld),
        .en(en),
        .pin(pin),
        .size_total(size_total),
        .size_filter(size_filter),
        .cout(cout),
        .pout(pout)
    );

    always begin
        #5 clk = ~clk;
    end

    initial begin
        clk = 0;
        rst = 0;
        ld = 0;
        en = 0;
        pin = 8'b0;

        size_total = 8'd64;
        size_filter = 4'd10;

        rst = 1; #10;
        rst = 0; #10;

        pin = 8'b0;
        ld = 1; #10;
        ld = 0; #10;

        en = 1;
        while (cout == 0) #10;
        en = 0;

        $stop;
    end

endmodule
```

```
module tb_filter_base_counter;

    parameter WIDTH = 4;

    reg clk, rst, ld, en;
    reg [WIDTH-1:0] pin;
    reg [WIDTH-1:0] filter_size;

    wire cout;
    wire [WIDTH-1:0] pout;

    filter_base_counter #(
        .WIDTH(WIDTH)
    ) uut (
        .clk(clk),
        .rst(rst),
        .ld(ld),
        .en(en),
        .filter_size(filter_size),
        .pin(pin),
        .cout(cout),
        .pout(pout)
    );

    always #5 clk = ~clk;
    initial begin
        clk = 0;
        rst = 0;
        ld = 0;
        en = 0;
        pin = 0;

        rst = 1;
        #10;
        rst = 0;
        #10;
        filter_size = 4'd2;
        en = 1;


        #10;

        #300
        $stop;
    end

endmodule
```

```
module tb_read_filter_counter;

    parameter SCRATCHPAD_SIZE = 16;
    parameter FILTER_SIZE_BIT = $clog2(SCRATCHPAD_SIZE);

    reg clk, rst, ld, en;
    reg [FILTER_SIZE_BIT-1:0] pin;
    reg [FILTER_SIZE_BIT-1:0] size_total;
    reg [FILTER_SIZE_BIT-1:0] size_filter;

    wire cout;
    wire [FILTER_SIZE_BIT-1:0] pout;

    read_filter_counter #(
        .SCRATCHPAD_SIZE(SCRATCHPAD_SIZE)
    ) uut (
        .clk(clk),
        .rst(rst),
        .ld(ld),
        .en(en),
        .pin(pin),
        .size_total(size_total),
        .size_filter(size_filter),
        .cout(cout),
        .pout(pout)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 0;
        ld = 0;
        en = 0;
        pin = 0;
        size_total = 5'd16;
        size_filter = 5'd5;

        rst = 1;
        #10;
        rst = 0;
        #10;
        pin = 4'd0;
        ld = 1;
        #10;
        ld = 0;
        en = 1;
        #300



        $stop;
    end

endmodule
```

Main Testbenches:

```
module testbench ();

    parameter DATA_WIDTH = 4, BUFFER_SIZE= 16, W_PARAM=1 , R_PARAM=1, SIZE_IFMAP= 7, SIZE_FILTER_SRAM = 5;
    parameter ADDR_WIDTH_FILTER = $clog2(SIZE_FILTER_SRAM);
    parameter ADDR_WIDTH_IFMAP = $clog2(SIZE_IFMAP);

    reg clk=0,rst=1,start=0, write_en_ifmap=0, write_en_filter=0;

    wire read_en_ifmap, read_en_filter, valid_ifmap, valid_filter, write_en_out, ready, stall;

    wire [DATA_WIDTH-1:0] inp_ifmap, inp_filter, out_top;
    reg [DATA_WIDTH-1:0] inp_buf_filter;

    reg [DATA_WIDTH+1:0] inp_buf_ifmap;

    wire [DATA_WIDTH+1:0] inp_ifmap_temp;

    reg [ADDR_WIDTH_FILTER-1:0] filter_size;

    reg [ADDR_WIDTH_IFMAP-1:0] stride;
    circular_buffer #(.DATA_WIDTH(DATA_WIDTH + 2), .BUFFER_SIZE(BUFFER_SIZE), .W_PARAM(W_PARAM), .R_PARAM(R_PARAM)) cb_ifmap
                    (.clk(clk), .rst(rst), .write_en(write_en_ifmap), .read_en(read_en_ifmap), .inp(inp_buf_ifmap),
                    .full(), .empty(), .ready(), .valid(valid_ifmap), .data_out(inp_ifmap_temp));

    circular_buffer #(.DATA_WIDTH(DATA_WIDTH), .BUFFER_SIZE(BUFFER_SIZE), .W_PARAM(W_PARAM), .R_PARAM(R_PARAM)) cb_filter
                    (.clk(clk), .rst(rst), .write_en(write_en_filter), .read_en(read_en_filter), .inp(inp_buf_filter),
                    .full(), .empty(), .ready(), .valid(valid_filter), .data_out(inp_filter));

    circular_buffer #(.DATA_WIDTH(DATA_WIDTH), .BUFFER_SIZE(BUFFER_SIZE), .W_PARAM(W_PARAM), .R_PARAM(R_PARAM)) cb_out
                    (.clk(clk), .rst(rst), .write_en(write_en_out), .read_en(), .inp(out_top),
                    .full(), .empty(), .ready(ready), .valid(), .data_out());

    wire end_signal, start_signal;

    assign {start_signal, end_signal, inp_ifmap} = inp_ifmap_temp;

    top #(
        .width(DATA_WIDTH),
        .SIZE_IFMAP(SIZE_IFMAP),
        .SIZE_FILTER_SRAM(SIZE_FILTER_SRAM)
    ) my_top (
        .clk(clk), .start(start), .ready(ready), .valid_ifmap(valid_ifmap), .valid_filter(valid_filter), .end_signal(end_signal),
        .filter_size(filter_size),
        .inp_buf_ifmap(inp_ifmap),
        .inp_buf_filter(inp_filter),
        .stride(stride),
        .out_buf(out_top),
        .read_en_filter_buf(read_en_filter), .read_en_ifmap_buf(read_en_ifmap), .write_en_buf(write_en_out), .stall(stall)
    );

    always begin #10;clk=~clk; end

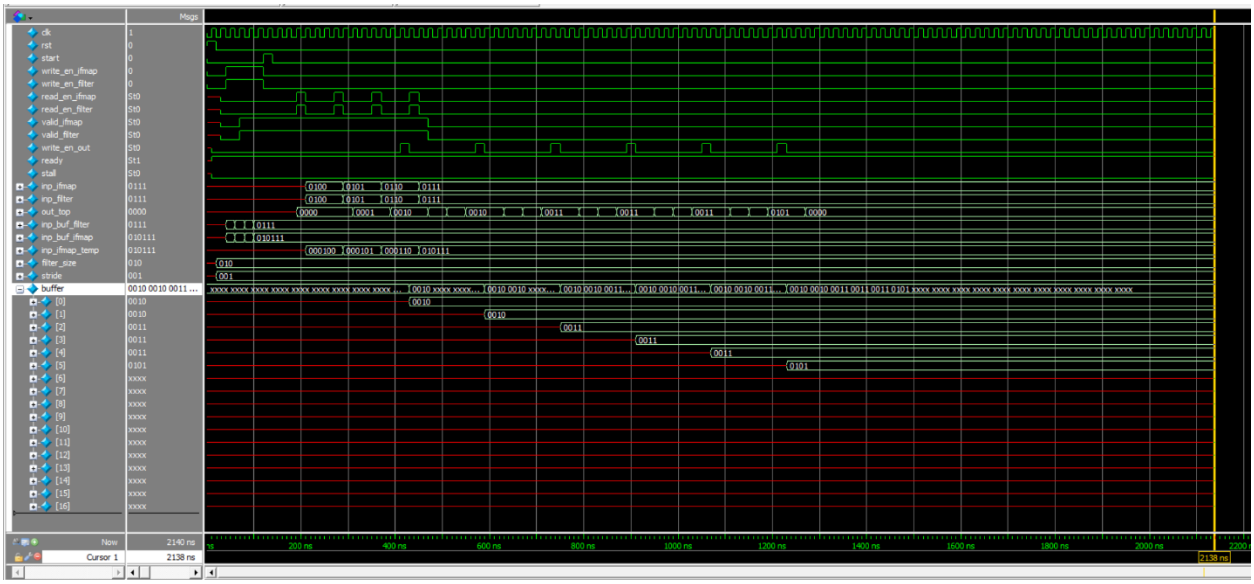
    initial begin
        #20 start=0; rst=0; write_en_ifmap=0; write_en_filter=0;
        filter_size = 2;
        stride = 1;
        #20
        write_en_ifmap = 1;
        write_en_filter = 1;
        inp_buf_filter = 4'b0100;
        inp_buf_ifmap = 6'b000100;
        #20
        write_en_ifmap = 1;
        write_en_filter = 1;
        inp_buf_filter = 4'b0101;
        inp_buf_ifmap = 6'b000101;
        #20
        write_en_ifmap = 1;
        write_en_filter = 1;
        inp_buf_filter = 4'b0110;
        inp_buf_ifmap = 6'b000110;
        #20
        write_en_ifmap = 1;
        write_en_filter = 1;
        inp_buf_filter = 4'b0111;
        inp_buf_ifmap = 6'b010111;
        #20

        write_en_ifmap = 0;
        write_en_filter = 0;

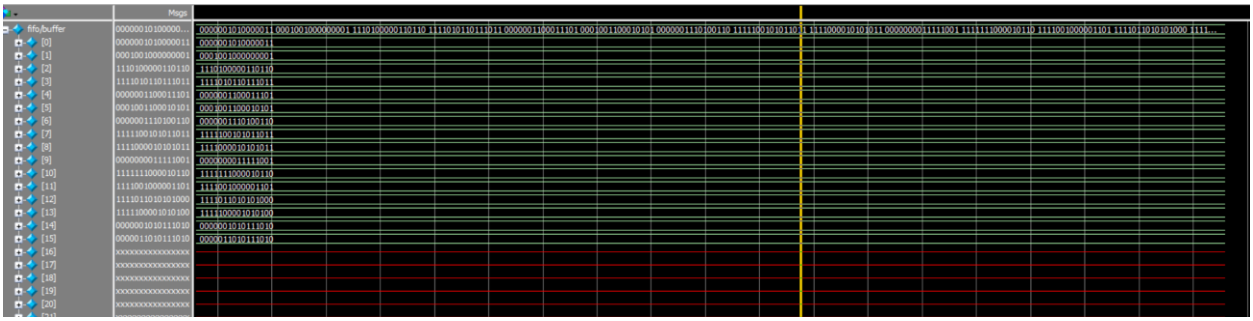
        start = 1;
        #20
        start = 0;
        #2000
        $stop;
    end

endmodule
```

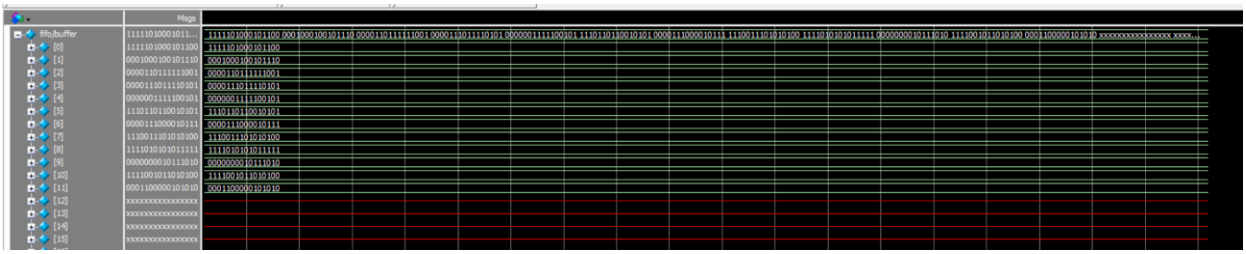
Wave Form:



TestBench1:



TestBench2:



TestBench5:

